

# Part B · 后端服务与数据层 (5 页)

**定位：**讲后端架构、运行时、预处理管道、多模数据库、跨域与反向代理。**架构口径**已从微服务切换为**模块化单体**：单一 FastAPI 进程 + 按业务域模块划分，统一入口 `backend/app/main.py` 监听 8000 端口。不含实操代码。

## 页面总览

| 页码   | 标题                  | 一句话                                            |
|------|---------------------|------------------------------------------------|
| B-P1 | 模块化单体架构与模块边界        | 为什么选单体 + 五大模块 + 共享层                            |
| B-P2 | 单体运行时与数据接入层         | Lifespan · APIRouter · paho-mqtt 独立线程          |
| B-P3 | DataProcessor 预处理管道 | 验证 → IsolationForest → 衍生指标                    |
| B-P4 | 多模数据库选型与三库职责        | TDengine + PostgreSQL/PostGIS/pgvector + Neo4j |
| B-P5 | 应用入口、跨域与反向代理        | 8000 单进程 · CORS · Vite/Nginx 双链路               |

## B-P1 · 模块化单体架构与模块边界

B-P1

模块化单体架构与模块边界

单一 FastAPI 进程：backend/app/main.py (:8000)

data  
接入  
+时序

station  
站点+  
MQTT绑

alert  
预警  
规则

report  
报告  
生成

ai  
引擎  
+智能

shared/ (db · mqtt · models · utils)

选型对比（为什么选单体，不选微服务）

| 维度    | 单体       | 微服务      |
|-------|----------|----------|
| 团队规模  | 小 · 直接受益 | 大 · 必须拆  |
| 运维复杂度 | 低 (1 进程) | 高 (N 进程) |
| 跨模块事务 | 函数调用     | 分布式事务    |
| 部署效率  | 单次       | 多次 + 编排  |
| 演进    | 可后续拆分    | 已拆       |

演进路径：清晰模块边界 + 共享层约束 → 保留拆微服务可行性

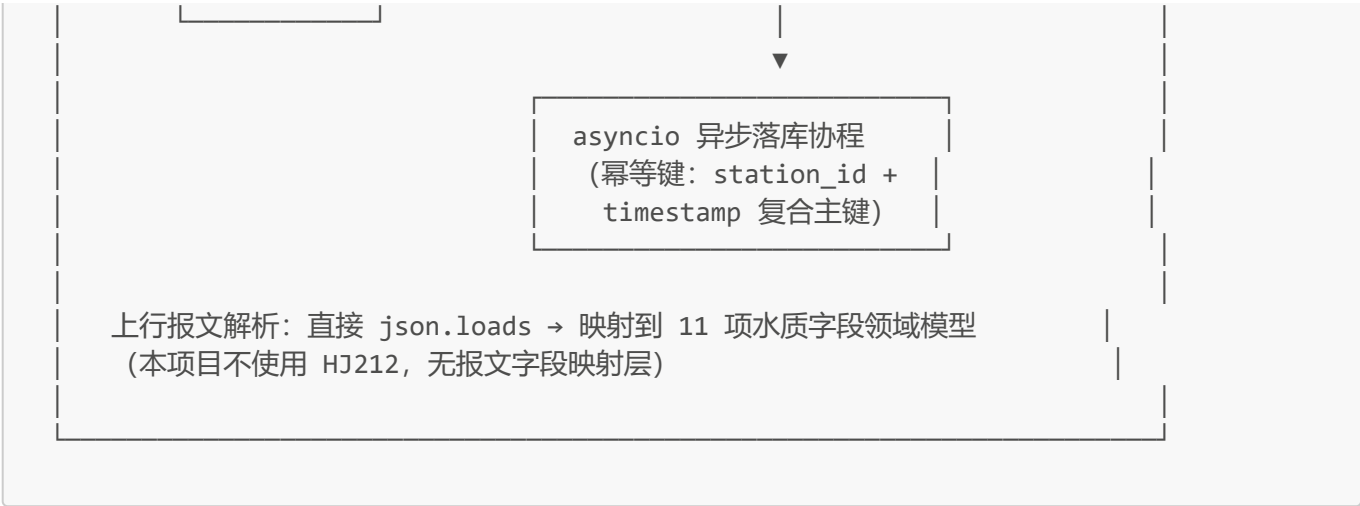
讲稿要点

- **架构口径**：项目已由原微服务（gateway + 5 个 service）整合为**模块化单体**——单一 FastAPI 进程内按业务域划分五个模块，统一部署、统一数据库会话池、统一生命周期
- **五大业务模块**：`app/data/`（MQTT 接入 + 时序落库） / `app/station/`（站点管理 + MQTT 绑定） / `app/alert/`（预警规则与触发） / `app/report/`（报告生成） / `app/ai/`（智能分析）
- **共享层** `backend/shared/`：数据库会话工厂、MQTT 客户端封装、跨模块领域模型、通用工具；所有模块复用同一套基础能力
- **选型理由**：当前团队规模与功能边界还不需要跨进程拆分；单体在**部署效率**、**跨模块事务**、**运维复杂度**上的优势对本项目更实际
- **演进设计**：每个模块保持独立的 `api/`、`services/`、`models/`、`database.py`，一旦某个模块需要独立扩容，按现有边界可直接切出为微服务
- **技术事实**：已弃用的 `backend/gateway/` 目录保留作多服务演进备选，但当前不纳入运行路径

**下一页过渡**：架构边界讲完了，下一步看这个进程的运行时是如何编排起来的。

B-P2 · 单体运行时与数据接入层





讲稿要点

- **Lifespan 统一启动钩子**: FastAPI 的 `@asynccontextmanager` 把五个模块的 DB 初始化、MQTT 订阅、模型预热按顺序串起来, 保证进程起来后所有子系统就绪
- **路由挂载**: `include_router(..., prefix="/api/v1")` 把六个 router 统一挂到版本前缀下; 依赖注入用 `Depends` 把 DB/TDengine/Neo4j 会话递给处理函数, 模块间不再各自管理连接
- **三种模块协作模式**: 事件驱动 (MQTT 收到消息触发)、同步调用 (REST 跨模块 API)、后台任务 (FastAPI BackgroundTasks 延后执行)
- **MQTT 订阅线程模型**: paho-mqtt 客户端运行在独立 IO 线程上, `on_message` 回调把消息塞进线程安全队列, asyncio 协程从队列取出做异步落库——避免阻塞事件循环
- **上行报文解析**: 直接 `json.loads` 把 MQTT payload 反序列化, 字段映射到系统领域模型的 11 项水质字段 (ph / do / temp / conductivity / turbidity / nh3\_n / tn / tp / codmn / chla / ...) ——因为上行已是自定义 JSON, **没有 HJ212 字段映射层**
- **幂等与回压**: 用 `station_id + timestamp` 作为复合主键去重; 高峰期启用本地队列 + 批量写入回压缓冲, 保护下游数据库

下一页过渡: 数据进来后不会直接写库, 先要过一层预处理管道。

B-P3 · DataProcessor 预处理管道





讲稿要点

- **验证层**：先做结构校验（必需字段是否齐全）、再做值域校验（pH ∈ [0, 14]、DO ≥ 0 等物理合理性）、再做类型强转与单位归一化；这一层是脏数据的第一道闸门
- **IsolationForest 原理**：基于「异常点在随机分裂树中更容易被隔离」的假设，用多棵随机树测量每个样本被隔离时的平均路径深度——异常点路径短、正常点路径长，归一化后输出 **anomaly\_score ∈ [0, 1]**
- **为什么这里做初筛**：后端入库前的无监督初筛开销小、免标注，能提前拦截明显异常；AI 引擎的 LSTM 重建误差判定留给**更复杂的时序模式**（D-P2）
- **衍生指标**：
  - **DO 饱和度**：根据 Henry 定律，用水温、盐度、气压补偿出当前实际氧饱和百分比
  - **TLI 富营养化指数**：以 TN、TP、Chla 等营养盐与藻类指标加权综合
  - **WQI 综合水质标识指数**：多指标按国标分级打分，输出一个 0~100 综合值给大屏呈现
- **数据质量分级**：valid / suspicious / invalid 三档——valid 正常走全路径；suspicious 仍落库但带标记，交由二次复核；invalid 进入死信队列 + 触发设备侧告警
- **技术事实**：这一层既是**数据清洗**也是**特征工程前置**——为 AI 引擎的时序分析和知识推理准备了可直接消费的语义字段

下一页过渡：预处理完的数据要分别沉到三种不同形态的数据库。

B-P4 · 多模数据库选型与三库职责

|      |              |
|------|--------------|
| B-P4 | 多模数据库选型与三库职责 |
|------|--------------|

为什么不用单一数据库：  
时序高频写 · 业务事务 + 空间 + 向量 · 关系遍历  
三类存储模型根本不同 → 多模各司其职

|                                                                                                                        |                                                                                                                                            |                                                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TDengine 3.2<br>(时序)                                                                                                   | PostgreSQL 16<br>+ PostGIS + pgvector                                                                                                      | Neo4j 5.15<br>(图谱)                                                                                                                                                 |
| 超级表 + 子表<br>站点 = 子表<br>TAGS:<br>station_id<br>type<br>basin<br>连续查询:<br>分钟/小时/日<br>保留策略:<br>原始 90 天<br>分钟 1 年<br>小时 永久 | 业务事务: stations /<br>alerts / reports<br>空间:<br>geography(Point, 4326)<br>+ GiST 空间索引<br>向量:<br>vector(16) + IVFFlat<br>Alembic 管 schema 迁移 | 节点:<br>Station<br>PollutionSrc<br>PollutionTy<br>Rule<br>EmergPlan<br>Department<br>关系:<br>FLOWS_TO<br>(distance,<br>travel_t)<br>HAS_RULE<br>HAS_PLAN<br>INVOLVES |
| ▲<br>水质时序写入                                                                                                            | ▲<br>业务 / 空间 / 案例向量                                                                                                                        | ▲<br>流域拓扑                                                                                                                                                          |

跨库一致性: 领域事件解耦 · 避免分布式事务  
初始化脚本:  
infrastructure/docker/postgres/init/\*.sql  
infrastructure/docker/neo4j/init/01~04.cypher

讲稿要点

- **单库做不到**：水质高频时序（每分钟 × 千站）、业务事务（站点/规则/报告）、空间查询（附近站点）、向量检索（相似案例）、图遍历（流域上下游）——存储模型差异太大，硬塞单一库会在任一维度都做不深
- **TDengine 3.2**：以「超级表 + 子表」组织，每个站点作为超级表下的一个子表，TAGS 列 (station\_id / type / basin) 支撑跨站聚合；连续查询做**分钟 / 小时 / 日级**降采样；**保留策略**：原始数据 90 天 / 分钟级 1 年 / 小时级永久，兼顾成本与历史可溯
- **PostgreSQL 16 + PostGIS + pgvector**：业务事务承载 station / alert / report 三张主表；**geography(Point, 4326) + GiST 空间索引**做附近站点、落域归属；**vector(16) + IVFFlat 索引**做历史案例向量检索；Alembic 管理 schema 迁移
- **Neo4j 5.15**：流域拓扑以图建模——节点有 Station、PollutionSource、PollutionType、Rule、EmergencyPlan、Department；关系中 **FLOWS\_TO** 携带 distance、travel\_time 属性用于溯源计算，**HAS\_RULE / HAS\_PLAN / INVOLVES** 把污染类型绑定到规则、预案和责任部门
- **跨库一致性**：不引入两阶段提交等分布式事务，而是用**领域事件解耦**（MQTT 消息 + 后台任务）保证最终一致；业务方关心的强一致都落在单库

- **技术事实：**初始化依赖 `infrastructure/docker/postgres/init/*.sql` 与 `infrastructure/docker/neo4j/init/01~04.cypher` 在容器首次启动时注入 schema 与种子数据

**下一页过渡：**数据库讲完，最后看单体怎么对外、前端怎么进来、MQTT 怎么走。

B-P5 · 应用入口、跨域与反向代理（真实配置）

B-P5

应用入口、跨域与反向代理（真实配置）

唯一对外入口：模块化单体 backend/app/main.py :8000  
(无独立网关进程前置；backend/gateway/ 已弃用，仅作演进备选)

路由注册 (include\_router · prefix="/api/v1")  
/api/v1/stations    /api/v1/data        /api/v1/mqtt  
/api/v1/alerts     /api/v1/ai         /api/v1/reports  
/health (健康检查)    /docs (OpenAPI)

跨域配置（真实）：  
CORSMiddleware  
  allow\_origins = ["\*"]  
  allow\_credentials / methods / headers = \*  
  （未配置白名单 · 未配置预检缓存 max\_age）  
演讲口径：演示阶段全开，生产再收敛

开发环境链路 (Windows · Vite)  
[前端 :5173] — /api /health → [http://127.0.0.1:8000]  
（强制 127.0.0.1 规避 localhost → ::1 的 socket hang up）

生产环境链路 (Nginx 反代)  
[浏览器] → [Nginx :80] -/api/→ [backend :8000]  
  limit\_req zone=api burst=20 nodelay  
  统一超时 · 转发头 · 通用 /ws/ 透传

MQTT 真实接入路径（不是前端直连 WebSocket）  
前端 — REST /api/v1/mqtt/\* → 后端 管理外部 MQTT 数据源配置  
后端 paho-mqtt — TCP 1883 → EMQX（内部订阅）  
数据落库 → HTTP/WebSocket → 前端 实时刷新  
未对外开放 EMQX 8083 (MQTT over WS)

讲稿要点

- **唯一对外入口：**模块化单体 `backend/app/main.py` 直接监听 8000 端口；**没有独立网关进程前置**；原 `backend/gateway/` 代码已合并入单体后弃用，仅作为多服务演进备选保留
- **路由真实结构：**六个 router 通过 `include_router(..., prefix="/api/v1")` 统一挂载  
——`/api/v1/stations`、`/api/v1/data`、`/api/v1/mqtt`、`/api/v1/alerts`、`/api/v1/ai`、`/api/v1/reports`；另暴露 `/health` 健康检查与 `/docs` OpenAPI

- **应用生命周期**：FastAPI `lifespan` 上下文按顺序初始化 `station / alert / report / data` 四个 PostgreSQL 库，并在 `data` 阶段加载 MQTT 连接配置启动后台订阅线程
- **跨域策略（真实）**：仅启用 `CORSMiddleware`, `allow_origins=["*"]` + `allow_credentials/methods/headers=*`，**未配置白名单与预检缓存**——演讲口径是「演示阶段全开，生产再收敛」
- **开发环境链路**：Vite `server.proxy` 将 `/api`、`/health` 转发到 `http://127.0.0.1:8000`——强制 `127.0.0.1` 规避 Windows 下 `localhost` 解析为 IPv6 `::1` 导致的 `socket hang up`
- **生产环境链路**：Nginx 反向代理 `/api/` → backend，并在 Nginx 层做限流 `limit_req zone=api burst=20 nodelay`、统一超时与转发头；通用 `/ws/` 路径透传给后端的实时推送通道
- **MQTT 真实接入路径**（不是前端直连 WebSocket）：
  - 前端通过 REST `/api/v1/mqtt/*` 管理「外部 MQTT 数据源」连接配置（`frontend/src/services/api.ts` 的 `mqttApi` 仅 HTTP 调用）
  - 后端 `paho-mqtt` 客户端走 TCP **1883** 订阅 EMQX，数据落库后再以 HTTP/WebSocket 形式下发给前端
  - Nginx 未对外开放 EMQX **8083** 的 MQTT-over-WebSocket 端口

**衔接到 Part C**：「后端把数据分别沉到了时序库、关系+空间+向量库和图库，对外以 `/api/v1/*` 暴露服务。接下来看前端是怎么把这些异构数据组织进一套可视化体系的。」